Tutoriais do Laboratorio Quantum IC

Andre Baeta

2026-05-20

Visao Geral

Este documento reune os tutoriais praticos do laboratorio `quantum_ic`. O objetivo e manter um guia unico para executar os exemplos em QCL, Qiskit, Cirq, PennyLane, OpenQASM, QML e a frente de zeros da zeta.

Preparar Ambiente

```
cd ~/quantum_ic
source .venv/bin/activate
```

Rodar Tudo

```
quantum_languages/scripts/run_qcl_examples.sh
quantum_languages/scripts/run_framework_examples.sh
python experiments/05_zeta_zeros_statistics.py --count 500 --
gue-size 500
python experiments/06_zeta_critical_line_scan.py --t-min 10 --
t-max 40
```

QCL

QCL e historico, mas excelente para estudar a forma procedural de algoritmos quanticos.

Rodar

```
cd ~/quantum_ic
bin/qcl quantum_languages/qcl/01_bell_pauli.qcl
```

Ou todos:

```
quantum_languages/scripts/run_qcl_examples.sh
```

Arquivos

```
01_bell_pauli.qcl
02_deutsch_balanced.qcl
03_grover_search.qcl
04_qft_phase_kick.qcl
05_diagonal_hamiltonian_phase.qcl
```

Quando usar

Use QCL para estudar a historia das linguagens quanticas, algoritmos procedurais e a relacao entre controle classico e registradores quanticos.

Qiskit

Qiskit e a melhor opcao do laboratorio para circuitos, simulacao com Aer, VQE, operadores de Pauli e ligacao com hardware IBM.

Rodar

```
cd ~/quantum_ic
source .venv/bin/activate
python quantum_languages/qiskit/algorithms_qiskit.py
```

Saidas

Os JSONs sao salvos em:

```
results/tables/quantum_languages/qiskit_*.json
```

Ideias de pesquisa

```
SparsePauliOp para Hamiltonianos
Statevector/Aer para simulacao
VQE para energia fundamental
QFT e estimacao de fase para espectros
```

Cirq

Cirq e bom para circuitos explicitos, controle fino de portas e estilo Google-like.

Rodar

```
cd ~/quantum_ic
source .venv/bin/activate
python quantum_languages/cirq/algorithms_cirq.py
```

Saidas

```
results/tables/quantum_languages/cirq_*.json
```

Quando usar

Use Cirq quando quiser construir circuitos de baixo nivel, estudar decomposicoes de portas e comparar simuladores de circuitos.

PennyLane

PennyLane e a melhor ponte entre circuitos quanticos e otimizacao/diferenciacao.

Rodar circuitos basicos

```
cd ~/quantum_ic
source .venv/bin/activate
python quantum_languages/pennylane/algorithms_pennylane.py
```

Rodar QML

```
python quantum_languages/qml/variational_classifier_pennylane.py
```

Saidas

```
results/tables/quantum_languages/pennylane_*.json
results/tables/quantum_languages/qml_variational_classifier.json
```

Quando usar

Use PennyLane para VQE variacional, QML, circuitos diferenciaveis, classificadores quanticos e modelos hibridos classico-quanticos.

BCS Quantum Project

Este tutorial descreve a integração do modelo BCS ao laboratório `quantum_ic`. A fundamentação teórica está disponível em `docs/BCS_QC.pdf` e a implementação prática está dividida entre `src/bcs.py`, `src/bcs_qiskit.py` e `quantum_languages/scripts/bcs_project.py`.

Execução

```
cd ~/quantum_ic
source .venv/bin/activate
python quantum_languages/scripts/bcs_project.py
python experiments/08_bcs_dynamics_trotter.py
python experiments/09_bcs_qiskit_trotter_circuit.py
python experiments/10_bcs_paper_observables.py
python experiments/11_bcs_pair_green_function.py
python experiments/12_bcs_gorkov_anomalous_green.py
python experiments/13_bcs_gorkov_nambu_matrix.py
python experiments/14_bcs_qiskit_shots_noise.py
```

Objetivo

Validar a correspondência entre:

- a formulação teórica do Hamiltoniano BCS em pseudospins;
- o mapeamento para operadores de Pauli em qubits;
- a estimativa da energia fundamental via VQE em diferentes frameworks.
- a dinâmica temporal exata contra Trotterização;
- a validação do circuito Qiskit Trotterizado contra a evolução matricial;
- observáveis físicos para paper: ocupações de pares, `C_ij=<b_i^\dagger b_j>`, autovalor condensado e proxy de gap efetivo.
- função de Green retardada de pares e espectro aproximado.
- função de Green anômala de Gor’kov e espectro anômalo aproximado.
- matriz de Gor’kov em base Nambu e traço espectral.
- medição por shots em Qiskit Aer, com e sem ruído.

Figuras e tabelas

```
results/tables/bcs_dynamics_trotter.csv
results/tables/bcs_commutator_norms.csv
results/tables/bcs_qiskit_trotter_circuit.csv
```

```
results/tables/bcs_paper_observables.csv
results/figures/bcs_trotter_error.png
results/figures/bcs_gap_condensate_time.png
results/figures/bcs_pair_occupations_time.png
results/figures/bcs_pair_correlation_exact_final.png
results/figures/bcs_pair_correlation_trotter_final.png
results/figures/bcs_pair_green_time.png
results/figures/bcs_pair_green_spectrum.png
results/figures/bcs_gorkov_anomalous_time.png
results/figures/bcs_gorkov_anomalous_spectrum.png
results/figures/bcs_gorkov_nambu_channels.png
results/figures/bcs_gorkov_nambu_spectral_trace.png
results/figures/bcs_qiskit_shots_noise_total_pairs.png
results/figures/bcs_qiskit_shots_noise_occupations.png
```

Roteiro de paper

Use `docs/bcs_paper_reproducibility.md` para reproduzir os parâmetros fixos, os comandos e os arquivos de saída principais.

Itens associados

```
docs/BCS_QC.pdf
docs/BCS_QC_comparacao.md
docs/bcs_paper_reproducibility.md
src/bcs.py
src/bcs_qiskit.py
quantum_languages/scripts/bcs_project.py
quantum_languages/tutorials/bcs.md
```

OpenQASM

OpenQASM e um formato portavel para descrever circuitos.

Arquivos

```
quantum_languages/openqasm/bell.qasm
quantum_languages/openqasm/grover_2q_mark_3.qasm
```

Quando usar

Use OpenQASM para portar circuitos entre ferramentas, registrar circuitos em texto e manter exemplos independentes de framework Python.

Observacao

O ambiente atual guarda os arquivos OpenQASM, mas a execucao principal esta nos frameworks Python e no QCL local.

Zeros da Zeta

Esta parte e classica/espectral, nao um circuito quantico direto.

Estatistica de espacamentos

```
        cd ~/quantum_ic
        source .venv/bin/activate
        python experiments/05_zeta_zeros_statistics.py --count 500 --
gue-size 500
```

Ou use a interface gráfica para variar parâmetros de forma interativa e ver o resultado diretamente:

```
        python experiments/zeta_explorer.py
```

Também é possível rodar uma busca aleatória dentro do strip crítico:

```
        python experiments/07_zeta_random_strip_search.py --sigma-min
0.01 --sigma-max 0.99 --t-min 10 --t-max 40 --samples 2000
```

Parâmetros:

- `--count 500`: número de zeros da função zeta usados para calcular espaçamentos. Começa no zero índice 1 por padrão.
- `--gue-size 500`: dimensão da matriz aleatória GUE usada para gerar espaçamentos de referência.

Gera:

```
results/tables/zeta_zeros_1_500.csv
results/tables/zeta_spacing_summary.csv
results/figures/zeta_spacing_histogram.png
```

Interpretação:

- O script compara os espaçamentos normalizados dos zeros da zeta com a estatística de espaçamentos de matrizes GUE.
- Se os dados da zeta seguirem a curva `GUE Wigner surmise`, isso indica que os zeros têm correlações semelhantes às de autovalores de matrizes hermitianas aleatórias.
- O arquivo `zeta_spacing_summary.csv` inclui métricas como média, variância e distâncias de Kolmogorov-Smirnov para Poisson e GUE.

Linha critica

```
        python experiments/06_zeta_critical_line_scan.py --t-min 10 --
t-max 40
```

Para buscar pontos pequenos de `ζ(s)` dentro do strip crítico fora da linha `σ = 1/2`:

```
        python experiments/07_zeta_random_strip_search.py --sigma-min
0.01 --sigma-max 0.99 --t-min 10 --t-max 40 --samples 2000
```

Parâmetros:

- `--t-min 10`: início do intervalo em `t` para avaliar `ζ(1/2 + it)`.
- `--t-max 40`: fim do intervalo em `t`.
- `--points`: número de pontos de amostragem entre `t-min` e `t-max` (padrão 900).
- `--zero-count`: quantos zeros da zeta computar para marcar no gráfico.

Gera:

```
results/tables/zeta_critical_line_scan.csv
results/figures/zeta_critical_line_scan.png
```

Interpretação:

- O gráfico mostra `|ζ(1/2 + it)|` ao longo do eixo crítico.
- Mínimos do módulo correspondem a zeros da função zeta no intervalo avaliado.
- As linhas verticais no gráfico indicam os zeros encontrados no mesmo intervalo.

Ponte com computacao quantica

O caminho de pesquisa natural e:

```
zeros da zeta -> espectro -> estatistica GUE -> hipotese Hilbert-Polya
Hamiltoniano H -> propagador exp(-iHt) -> fases -> QFT/estimacao de
fase
```

Nesta fase, o codigo calcula os dados espectrais e compara com GUE. O proximo passo e construir Hamiltonianos toy cujos espectros imitem estatisticas GUE.